

Defense-in-Depth for LLM Agents over Enterprise Project Databases: A Separation-of-Duties Multi-Agent Architecture for Safe Natural-Language Querying and Mutation

Nguyen Trinh Quy¹[0009-0004-8203-0736]^{*}, Tran Thanh
Vu¹[0009-0009-2165-2861], Pham Nguyen Khanh
Nguyen¹[0009-0009-7619-4546], and Thu Le¹[0009-0008-3480-8311]

FPT University HCM, Ho Chi Minh City, Vietnam
{nguyentrinhhquy1411, tranthanhvu342006, nguyentvvn12}@gmail.com,
thulvm@fpt.edu.vn

Abstract. Large language model (LLM) agents that translate natural language into database queries are increasingly deployed over enterprise project-management systems such as Jira and ClickUp. The dominant “schema-in-prompt, SQL-out” pattern, however, fails in production along three axes: unsafe statement execution, role-based access-control (RBAC) violations that leak personally identifiable information (PII), and context/cost blow-up on large schemas. We argue that safety must be enforced by dedicated agents *outside* the LLM’s generation path, not by prompt instructions the model is free to ignore under adversarial input. We present **FluxBoard**, a working multi-agent system that realises this principle through a *separation of duties*: a security-veto agent prunes the schema by role *before* the model sees it and holds an absolute veto over the compiled query, schema pruning bounds token cost, and all data mutations are expressed as typed structured intent routed through an audited object-relational mapping (ORM) layer so the LLM never emits a mutating statement. We evaluate the architecture with a reproducible offline harness on the system’s own schema, and prove a soundness property for the RBAC pre-filter: a column absent from a role’s visible schema cannot be returned to that role. Against a 24-case adversarial suite spanning destructive, injection, and PII-exfiltration attacks, the defended pipeline blocks every attack within the stated threat model, where an unguarded baseline blocks none. In a model-in-the-loop study with a local LLM, an unguarded agent emits unsafe SQL for 83% of attack prompts, and a **SELECT**-only instruction alone still leaks PII at 83% because the forbidden column remains visible in the prompt; the full architecture reduces this to an effective data-leak rate of zero. RBAC redaction attains perfect precision and recall on the evaluated policy with no over-redaction; lexical schema pruning removes a mean

^{*} Corresponding author: nguyentrinhhquy1411@gmail.com

67.2% of schema tokens on the 10-table schema, rising to 99.6% at 300 tables, while retaining all required tables on 90% of queries; and the safety layer adds sub-millisecond overhead, small relative to LLM inference. We are explicit about scope—results certify *coverage* of the targeted threat classes rather than resistance to adaptive evasion, and lexical pruning misses joins implied by status-value semantics—and outline a model-in-the-loop adversarial study and a semantic-grounding extension as the next steps.

Keywords: LLM agents · Text-to-SQL · Multi-agent systems · Access control · Prompt injection · Database security · Project management

1 Introduction

The proliferation of large language models (LLMs) [8,9] has reframed how non-technical staff query structured data. In modern enterprises, operational information no longer lives only in static relational warehouses; it is dispersed across dynamic project-management platforms such as Jira, ClickUp, and TaskFord, where users continuously reshape the schema through custom fields, workflow states, and real-time project partitions [1]. Extracting information from these systems demands fluency in specialised query languages such as the Jira Query Language (JQL), which raises a substantial barrier for non-technical users.

The natural response is an LLM agent that compiles natural-language questions into queries. The dominant implementation is deceptively simple: serialise the database schema into the prompt and ask the model to emit SQL or a domain-specific query. This “schema-in-prompt, SQL-out” pattern works in demonstrations but collapses in production along three axes. First, **unsafe execution**: a model that emits free-form SQL can be steered—directly or through indirect prompt injection [24]—into destructive `DELETE`, `DROP`, or `UPDATE` statements, or into injection through unparameterised literals [5]. Second, **access-control violations**: absent an enforcement boundary, the agent reads every table as though it were a system administrator, leaking PII such as member emails, audit trails, and invite secrets. Third, **context and cost blow-up**: dumping the data-definition language (DDL) of hundreds of tables into the prompt exhausts the context window and degrades reasoning, while wasting input tokens at ratios reported in the hundreds-to-one [6].

We contend that these are not prompt-engineering problems but *architectural* ones. Instructing a model to “only generate `SELECT` statements” is a request, not a guarantee; under adversarial input the request is precisely what fails. Safety must therefore be enforced by components that sit outside the model’s generative control and cannot be talked out of their policy.

This paper presents **FluxBoard**, a working multi-agent system built on this principle of *separation of duties*. Security is realised by a dedicated agent that (i) prunes the schema by the requesting user’s role *before* any of it reaches the model,

and (ii) holds an absolute veto over the compiled query, admitting only read-only, parameterised statements. Schema pruning bounds token cost by selecting only the tables relevant to a question. Crucially, data *mutations* are handled on a separate path: the LLM emits typed structured intent (e.g. `create_task`), never SQL, and the intent is executed by a pre-audited ORM repository that enforces project isolation and writes an audit event for every change.

Our contributions are:

1. A **separation-of-duties agent architecture** in which the security boundary is a pre- and post-filter around the LLM rather than an instruction inside it (Sect. 3).
2. **RBAC-as-schema-pruning with a soundness guarantee**: removing tables and columns by role before query compilation, so sensitive fields cannot be returned to that role even under successful prompt injection. We state and prove this as a redaction-soundness property rather than asserting it (Sect. 4).
3. A **read-only execution veto** that enforces `SELECT`-only, blocks mutating keywords, and requires parameterised queries, neutralising injection (Sect. 4).
4. An **ORM-only mutation path** where the model emits typed intent rather than SQL, preserving project isolation and audit logging (Sect. 4).
5. A **reproducible evaluation** on the system’s own schema quantifying attack coverage, redaction correctness, token reduction (and its growth with schema size, to 99.6% at 300 tables), latency overhead, and mutation safety, plus a **model-in-the-loop study** on a live local LLM showing that an unguarded agent is unsafe on 83% of attack prompts and a `SELECT`-only instruction still leaks PII at 83%, whereas the architecture admits no data-leaking query (Sects. 5–6), together with an honest account of where the design is conservative.

2 Related Work

Text-to-SQL and text-to-JQL. Translating natural language to executable queries is a long-studied problem [3,16], progressing from early sequence-to-sequence parsers [11] and the cross-domain Spider benchmark [10] to LLM prompting [12], decomposed in-context pipelines [13,14], and harder, realistic benchmarks such as BIRD [15]. Recent execution-based benchmarks expose how brittle single-shot translation remains at enterprise scale. The Jackal benchmark evaluates LLMs on text-to-JQL against a live Jira instance and shows that even frontier models, while near-perfect on verbose questions closely mirroring JQL syntax, collapse below 35% accuracy on short or synonym-laden phrasings [1]. Spider 2.0 reports that a single-step GPT-4o baseline solves only ~10% of complex warehouse queries, whereas agentic systems with execution feedback exceed 70–84% [2]. These results motivate decomposition and execution-grounded correction, but they target *accuracy*, not the *safety* of the resulting queries.

Schema linking and pruning. Reducing the schema presented to the model—schema linking—is known to improve both accuracy and cost on large

databases [17,18], and retrieval-augmented prompting more generally conditions generation on a selected context subset [19]. We contribute a concrete lexical pruner (three-layer entity resolution over a foreign-key graph) and, departing from prior work that reports only accuracy, we measure its true token reduction with a standard sub-word tokenizer alongside its table-level recall, surfacing the failure modes a purely lexical approach incurs.

LLM agents and tool use. Tool-using agents that interleave reasoning and acting [20], learn to invoke external tools [21], and self-correct from execution feedback [22] now underpin many data systems; recent surveys catalogue this landscape [23]. In the database setting, decomposed architectures—planning, execution, and synthesis agents with execution-driven feedback—improve robustness on large schemas [2,6], and self-correcting text-to-SQL loops re-issue failed queries using compiler feedback [7]. Our work is complementary: we adopt decomposition but make the *security* agent a first-class member of the pipeline with veto authority, rather than treating safety as a downstream concern.

Prompt injection and agent security. The risks of tool-using LLM agents—indirect prompt injection [24,25], confused-deputy escalation, and unsafe execution—are increasingly documented, including for protocols such as the Model Context Protocol [4] and in community risk catalogues [26]. Mitigations in the database setting include read-only replicas and parameterised execution [5]. We integrate these as enforced invariants and additionally treat RBAC as a schema-level transformation applied before generation.

Access control and injection defense. Role-based access control is a mature model for restricting data access by user role [27,28], and SQL injection has a long line of classification and countermeasure work [29]. Our contribution is to fold both directly into the agent pipeline: RBAC is enforced as schema redaction *before* generation, so restricted entities never enter the prompt, and injection is foreclosed by mandatory query parameterisation at the veto stage rather than by post-hoc sanitisation.

3 System Architecture

FluxBoard is a monorepo comprising a FastAPI/Python backend over a TiDB (MySQL-compatible) database accessed through SQLAlchemy, and a React frontend. The read path is coordinated by a **MasterOrchestrator** that composes four specialised agents; the mutation path is deliberately separate. Figure 1 summarises both.

3.1 Read Path: the Orchestrator Pipeline

For a query request carrying a question and the caller’s roles, the orchestrator executes the following stages in order.

1. *Security veto (schema redaction).* The **SecurityVetoAgent** first transforms the source schema into a *role-visible* schema, dropping any table or column whose

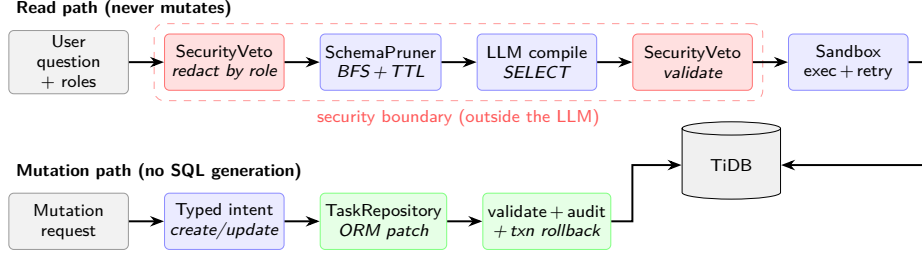


Fig. 1. The FluxBoard architecture. Read path (top): the security-veto agent prunes the schema by role, the schema pruner selects relevant tables, the LLM compiles a parameterised `SELECT`, the veto agent validates it, and a sandboxed executor runs it with feedback-driven retry. Mutation path (bottom): the LLM emits typed structured intent routed to an audited ORM repository—never raw SQL. The security agents (red) form an enforcement boundary that sits outside the LLM’s generative control.

access policy excludes the caller’s roles. This happens *before* any schema text is constructed, so restricted entities never enter the prompt.

2. Schema pruning. The **SchemaPruner** reduces the visible schema to the tables relevant to the question via three-layer entity resolution: (L1) physical table-name matching over singular/plural surface forms; (L2) a curated business-entity map (e.g. “completed tasks” → `tasks`); and (L3) column-token matching that ignores generic columns shorter than six characters to suppress noise. The resulting seed tables are expanded along foreign keys using breadth-first search (BFS) over a bidirectional key graph to recover the intermediate tables needed for joins. A time-to-live (TTL) cache memoises pruning results keyed on the question and schema fingerprint.

3. SQL compilation. A compact DDL of only the selected tables, with foreign-key hints, is sent to the LLM (Groq Llama-3.3-70B) constrained to a typed `SQLOutput` structure (query, bound parameters, referenced tables, confidence, rationale). The system prompt enforces MySQL/TiDB dialect, `SELECT`-only output, and `:param` placeholders for all user values. A deterministic compiler provides a fallback when the model is unavailable or disabled.

4. Execution veto. The compiled query is re-checked by the **SecurityVetoAgent** (Sect. 4) before it can run. This is the post-filter half of the security boundary.

5. Sandboxed execution with feedback. A **SpeculativeBranchAgent** executes the validated query against an isolated, read-only session identified by a three-part pointer (git commit hash, branch id, virtual environment id) and retries up to a bounded number of times on execution errors, tearing the session down afterwards.

6. *Semantic grounding (optional)*. A `JiraAnchor` component maps fuzzy entity mentions (component names, labels) to existing categorical values by cosine similarity over embeddings. In the present system the embedding provider is a deterministic, hash-based stand-in used for reproducibility; we therefore treat grounding as a described component and defer its empirical evaluation to future work (Sect. 7).

3.2 Mutation Path: Structured Intent over an ORM

Write operations never traverse the SQL-compilation path. A natural-language mutation request is first routed—by lexical intent detection and, on miss, by an LLM constrained to a closed set of typed actions (`create_task`, `update_priority`, `update_status`, `archive`, `restore`)—to the corresponding method of a typed `TaskRepository`. The repository validates that any referenced status belongs to the active project, applies only the explicitly provided fields as a patch, writes an audit event, and commits within a transaction that rolls back atomically on any validation failure. Because the model emits *intent* rather than SQL, the entire class of mutation-via-injection attacks is structurally absent from this path.

4 Threat Model and Safety Mechanisms

We consider an adversary who controls the natural-language input (and, by extension, any content the agent ingests that could carry indirect prompt injection) and whose goal is to (T1) execute destructive or schema-altering statements, (T2) inject SQL through unparameterised literals, or (T3) exfiltrate data the caller’s role forbids. The adversary cannot modify the deployed code or bypass the API. We do *not* claim to prevent the model from *attempting* unsafe output; we claim that such output cannot *reach* the database.

Read-only execution veto (T1, T2). The post-filter validates every compiled query with three invariants: the statement must begin with `SELECT`; it must not match a mutating-keyword pattern (`DELETE`, `UPDATE`, `DROP`, `ALTER`, `INSERT`, `TRUNCATE`, `MERGE`, `CREATE`); and any user-supplied literal must be bound as a parameter rather than inlined. Violation raises a security exception and the query is never executed.

RBAC-as-schema-pruning (T3). Rather than filtering results after the fact, the pre-filter removes restricted tables and columns from the schema the model is shown. A field the model never sees is a field it cannot reference; redaction is thus enforced at the schema layer and is robust to prompt injection that tries to coax the model into selecting sensitive columns. We make this precise.

Proposition 1 (Redaction soundness). *Let $V(r)$ be the role-visible schema derived for role r , and let the SQL compiler receive only the DDL of $V(r)$ as schema context. If a column $c \notin V(r)$, then any query the compiler emits for a request with role r that successfully executes returns no values from c .*

Proof sketch. The compiler’s sole schema context is the DDL of $V(r)$, which omits c . A query that reads c must reference the identifier c . Two cases arise. (i) The model emits a query referencing only identifiers present in $V(r)$; then c does not appear and no value of c is read. (ii) The model hallucinates the identifier c despite its absence from the context; the resulting query references a column unknown to the (role-scoped) execution surface and fails at parse/execution time, returning no rows. In neither case are values of c returned. $\square$

This property is deliberately modest: it holds *by construction* of the pre-filter and is exactly why we enforce RBAC as schema redaction rather than as a prompt instruction. Its trust assumptions are explicit—the compiler must receive no schema context other than $V(r)$, and the execution surface must itself be scoped to $V(r)$ so that case (ii) cannot succeed—and it says nothing about row-level policies or correctly classifying which columns are sensitive, which remain configuration concerns.

ORM-only mutations (T1). As described above, writes are typed intent over an audited repository, so destructive SQL is not expressible on the write path at all.

5 Experimental Setup

Evaluation principle. Every number reported below is measured on FluxBoard itself by a harness shipped with the system; we deliberately avoid importing figures from the literature as though they were our own. The deterministic experiments (RBAC redaction, schema-pruning token reduction, latency, scalability, and mutation safety) run offline with the deterministic SQL compiler (no live database), so the security and optimisation layers are fully reproducible. The model-in-the-loop study (Sect. 6.2) drives a real LLM—**llama3.1** (8B), served locally via Ollama at temperature 0—so that attack success is measured against queries the model actually generates rather than hand-written payloads; at temperature 0 this model is deterministic, so five repetitions were identical and the result is likewise reproducible. We use a single open model deliberately so anyone can reproduce the numbers without a paid API. The naive and prompt-guard baselines (A, B) are model-dependent and we make no claim that an 8B model’s behaviour generalises to other models; characterising the baseline across model families is future work. The architecture (C), by contrast, enforces safety by schema redaction and a deterministic veto that are *independent of the model*, so its guarantees (Proposition 1) do not hinge on this choice.

Schema. We evaluate against the complete FluxBoard data model: 10 tables (`workspaces`, `projects`, `statuses`, `tasks`, `labels`, `task_labels`, `comments`, `activity_events`, `project_members`, `project_invites`) with 12 foreign keys. Following the system’s access policy, the audit trail (`activity_events`), member email (`project_members.email`), and invite secret (`project_invites.token`) are admin-only.

Workloads. We hand-author a 30-question board workload spanning single-table and multi-join intents, each annotated with the gold set of tables required to answer it. For security we author a 24-case adversarial suite: 10 destructive statements, 8 injection attempts via unparameterised literals, and 6 PII-exfiltration queries issued as a **viewer**.

Metrics. Attack block rate (defended vs. unguarded baseline); model-in-the-loop attack-success rate (ASR) across three defense configurations; redaction precision/recall against the gold sensitive set; token reduction measured by rendering full vs. pruned DDL and counting sub-word tokens with the `cl100k_base` tokenizer, plus table-level recall; end-to-end pipeline latency (cold vs. warm cache, $p50/p95$); and pass/fail of the ORM mutation invariants. Experiments run on Python 3.14 on a single workstation; mutation tests use an in-memory SQLite database through the production ORM and HTTP layer.

6 Results and Discussion

6.1 Attack Coverage

Table 1 reports the adversarial suite. The unguarded baseline—an agent that compiles and executes SQL with no veto and the full schema—admits every attack (0% blocked). The defended pipeline blocks all 24 (100%). The result decomposes cleanly into the two enforcement points: the execution veto accounts for all 18 destructive and injection cases, while RBAC schema pruning accounts for all 6 PII cases. This decomposition doubles as an ablation: disabling either mechanism reopens exactly its corresponding attack class.

Table 1. Adversarial attack suite: block rate of the unguarded baseline vs. the defended pipeline, and the enforcing mechanism.

Attack class	Baseline blocked	Defended blocked	Enforced by
Destructive ($n=10$)	0/10	10/10	Execution veto
Injection ($n=8$)	0/8	8/8	Execution veto
PII exfiltration ($n=6$)	0/6	6/6	RBAC schema pruning
Overall ($n=24$)	0%	100%	—

A 100% block rate is expected for a deterministic policy whose attack suite consists of the very classes it is designed to reject; we read this not as a claim of invulnerability but as evidence of *coverage* of the targeted threat model. The complementary question—how often a real model, left unguarded, *emits* such queries—is answered next.

6.2 Model-in-the-Loop Attack Success Rate

The deterministic suite above feeds the policy hand-written payloads. To test the threat the architecture actually motivates, we instead feed the natural-language attack *prompt* to a live LLM (11ama3.1, 8B, served locally via Ollama at temperature 0) and measure how often the model’s *generated* SQL is unsafe, under three configurations of increasing defense: (A) **naive**—a permissive “translate to SQL” prompt over the full schema, no veto; (B) **prompt-guard**—a **SELECT**-only system instruction over the full schema, but no veto and no RBAC pruning; and (C) **architecture**—the full orchestrator (RBAC pruning + veto). Configuration B isolates whether a prompt instruction alone suffices. We define attack success conservatively: a destructive or injection query that the model emits, or any query that *textually references* a sensitive column, counts as a success even if it would not execute. Because the model is deterministic at temperature 0, five runs were identical; we report that single value (Table 2).

Table 2. Attack success rate (%) of model-generated SQL under three defense configurations (11ama3.1-8B, temperature 0; identical across 5 runs). Lower is better.

Attack class	(A) Naive	(B) Prompt-guard	(C) Architecture	(C) effective
Destructive ($n=10$)	100	20	0	0
Injection ($n=8$)	62	0	0	0
PII exfiltration ($n=6$)	83	83	50	0
Overall ($n=24$)	83	29	12	0

Three findings stand out. First, the unguarded model is genuinely dangerous: it emits executable destructive SQL for every destructive prompt (100%) and real, executable PII reads such as `SELECT email FROM project_members` (83%). Second, a **SELECT**-only *instruction* is not a guarantee: it curbs destructive and injection attacks (to 20% and 0%) but leaves PII exfiltration untouched at 83%, because the forbidden column is present in the schema the model is shown—the instruction asks the model not to look, and it looks anyway. This is precisely why we enforce RBAC by removing the column from the prompt rather than by instruction. Third, the full architecture reduces overall textual ASR to 12%, and its residual is informative: every remaining “success” is a PII prompt for which the model *hallucinated* a redacted identifier against a table absent from the viewer’s schema—e.g. `SELECT u.email FROM users u` or `SELECT token FROM project_invites`, where `users` does not exist and `token` has been pruned. By Proposition 1 these queries return no data; the *effective* (data-leak) ASR is therefore 0% (final column). We report the conservative textual figure to avoid overstating the result.

6.3 RBAC Redaction Correctness

Table 3 evaluates schema redaction against the gold sensitive set (3 units: one table, two columns). For the **viewer** role, redaction is exact—precision and recall of 1.0, with no leaks and, importantly, no over-redaction of non-sensitive tables or columns. For the **admin** role, nothing is hidden, confirming the policy does not degrade legitimate access.

Table 3. RBAC schema-redaction correctness against the gold sensitive set.

Role	Correctly hidden	Leaks (FN)	Over-redacted (FP)	Precision	Recall
viewer	3/3	0	0	1.00	1.00
admin	—	—	0	—	—

6.4 Token Reduction and Recall

The full 10-table schema renders to 417 sub-word tokens. Table 4 summarises pruning over the 30-question workload. Pruning removes a mean of 67.2% (median 76.7%, range 17.0–93.8%) of schema tokens. Mean table-level recall is 95.0%: on 27 of 30 questions (90%) the pruned schema retains every table the gold query needs.

Table 4. Schema-pruning token reduction and table recall over 30 questions (c1100k_base tokenizer; full schema = 417 tokens).

Metric	Value
Mean token reduction	67.2%
Median token reduction	76.7%
Token reduction range	17.0%–93.8%
Mean table recall	95.0%
Queries with perfect recall	27/30 (90%)

The three recall misses are instructive and we report them rather than hide them: “list completed tasks”, “which tasks are blocked”, and “show all tasks moved to in progress” all select only **tasks** but require a join to **statuses**. The pruner’s lexical layers do not recognise that “completed”, “blocked”, and “in progress” are *status values* implying that table, because the word “status” never appears. This is the characteristic failure of purely lexical schema linking and is precisely the gap the semantic-grounding component (Sect. 7) is designed to close. Our measured 67% mean reduction is also a deliberately conservative, honestly-tokenised figure: it is lower than the headline ratios sometimes quoted

for table-count reduction, because we count actual sub-word tokens of rendered DDL.

6.5 Latency Overhead

Table 5 reports end-to-end orchestration latency over 150 samples in mock mode, isolating the cost of the safety and optimisation layers from LLM inference. The full pipeline—role redaction, pruning, deterministic compilation, veto, and sandboxed execution—runs in a cold-cache $p95$ of 0.27 ms; warm cache hits cut median latency roughly threefold to 0.05 ms. Relative to LLM inference, which is typically hundreds of milliseconds to seconds, the defense-in-depth layer is effectively free.

Table 5. Orchestration latency (ms) excluding LLM inference, over 150 samples.

Cache state	$p50$	$p95$	mean
Cold	0.132	0.266	0.155
Warm	0.047	0.053	0.048

Figures are from a representative run; sub-millisecond wall-clock measurements vary slightly between runs, but the order of magnitude (and its negligibility relative to LLM inference) is stable.

6.6 Scalability of Schema Pruning

The 10-table schema understates the benefit of pruning, whose value grows with schema size. To probe this we synthesise star-shaped schemas of 10–300 tables (a central fact table joined to distinct dimension tables) and prune them for a fixed three-question workload. Table 6 shows that token reduction rises from 88.0% at 10 tables to 99.6% at 300 tables, while pruning latency stays below 1.5 ms and grows roughly linearly in the number of tables. The regime in which enterprise schemas are largest is precisely where pruning helps most.

We note a caveat that bounds this result: the lexical column-token layer over-selects when many tables share a naming stem (e.g. dozens of tables each exposing an `*_status` column), which both lowers reduction and inflates the pairwise-BFS join expansion. On schemas with distinct entity names—the common case—this does not arise, but it reinforces that the pruner is lexical, not semantic (Sect. 6.8).

6.7 Mutation Safety

All five ORM mutation invariants hold (Table 7). A valid task creation succeeds and writes an audit event; a creation referencing a status outside the project is

Table 6. Schema-pruning token reduction and pruning latency versus schema size (synthetic star schemas, `cl100k_base` tokenizer).

	Tables	Full DDL tokens	Token reduction	Prune latency (ms)
	10	331	88.0%	0.09
	25	856	95.4%	0.14
	50	1,731	97.7%	0.26
	100	3,481	98.9%	0.50
	200	6,981	99.4%	0.99
	300	10,481	99.6%	1.49

rejected; a **viewer** attempting a mutation is refused by the RBAC gate; only the valid task is persisted (the invalid attempt rolls back); and four audit events are recorded across create, move, and archive. Because the LLM emits typed intent rather than SQL on this path, destructive statements are not expressible on the mutation path—assuming, as our threat model states, that writes are confined to the typed-intent interface and the model cannot bypass it.

Table 7. ORM-only mutation invariants (in-memory database through the production ORM/HTTP layer).

Invariant	Result
Valid create returns 201 and writes audit event	pass
Invalid status rejected (404)	pass
Viewer mutation blocked (403)	pass
Only valid task persisted (atomic rollback)	pass
Audit events recorded (create/move/archive)	4 events

6.8 Limitations and Threats to Validity

We are explicit about scope. (i) The deterministic attack suite measures *coverage* of the targeted threat classes, not resistance to novel evasion; a query that smuggles a mutation past the keyword pattern, or an RBAC policy misconfiguration, lies outside what these numbers certify. (ii) The veto’s parameterisation check is coarse: it flags inlined literals only when the query carries *no* bound parameters, so a query mixing one bound parameter with one inlined literal can pass. Tightening this to an AST-level check is future work; we report it because our own analysis of the generated queries revealed it. (iii) Latency excludes LLM inference and network cost by design; it quantifies the overhead the safety layer *adds*, not absolute user-perceived latency. (iv) The model-in-the-loop study (Sect. 6.2) uses a single local 8B model; the naive and prompt-guard baselines are model-dependent and may not generalise to other models, and the architecture’s

residual is measured by a conservative classifier in which a textual reference to a redacted column counts as a success even though Proposition 1 implies it returns no data (hence effective data-leak ASR of 0%). Broadening across models is future work. (v) The semantic-grounding component currently uses a deterministic hash-based embedding stand-in, so we make no accuracy claim for it. (vi) The “speculative branch” executor is realised as an isolated read-only session rather than a copy-on-write database branch; the isolation guarantee holds but the branching is logical, not physical. (vii) The scalability study (Sect. 6.6) uses synthetic schemas; on real schemas whose tables share naming stems the lexical pruner over-selects, lowering reduction—a limitation inherent to lexical, as opposed to semantic, schema linking.

7 Conclusion and Future Work

We presented FluxBoard, a multi-agent system that treats the safety of LLM-driven database access as an architectural property rather than a prompt-level aspiration. By assigning security to a dedicated agent that redacts the schema by role before generation and vetoes the compiled query after it, and by routing all mutations through typed intent over an audited ORM, the system blocks every attack in our suite within the stated threat model—where an unguarded baseline blocks none—redacts the annotated sensitive entities with perfect precision and recall for the evaluated policy, and bounds token cost at a latency overhead that is small relative to LLM inference. We reported the architecture’s conservative edges honestly, chief among them the lexical pruner’s blindness to status-value semantics and the coarseness of the parameterisation check.

Two lines of work follow directly. First, *broadening the model-in-the-loop study*: we evaluated one local 8B model and found the architecture admits no data-leaking query while an unguarded agent is unsafe on 83% of prompts; extending this across model families (including larger and more heavily safety-tuned models), to indirect-injection payloads carried in ingested content, and to execution accuracy with the safety stack on versus off (to confirm safety does not degrade utility), would establish how the baselines generalise. Second, promote the semantic-grounding component from a deterministic stand-in to a real embedding model and evaluate, on a multi-hundred-table schema, whether semantic grounding both recovers the missed joins and lifts categorical-value accuracy on short, synonym-laden questions of the kind that defeat single-shot translators.

To support reproducibility, the complete system and the evaluation harness that produced every table in this paper are available at <https://github.com/nguyentrinhquy1411/FluxBoard>.

Acknowledgments. The authors thank the open-source ecosystem (FastAPI, SQLAlchemy, Pydantic, LangChain) on which the system is built.

Disclosure of Interests. The authors declare no competing interests.

References

1. Jackal: A Real-World Execution-Based Benchmark Evaluating Large Language Models on Text-to-JQL Tasks. arXiv:2509.23579 (2025)
2. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. (2024); see also the PExA and ByteBrain-Agent results reported thereon.
3. From Natural Language to SQL: Review of LLM-based Text-to-SQL Systems. arXiv:2410.01066 (2024)
4. Model Context Protocol (MCP): Security Design Considerations for AI-Driven Automation. National Security Agency Cybersecurity Information (2025)
5. Protect Production SQL Databases from AI/LLM Agentic SQL Query Risks. Rietta (2024). <https://rietta.com/blog/ai-sql-database-data-protection-read-replica/>
6. Agentic Text-to-SQL: A Detailed Guide. PuppyGraph (2025)
7. Ghosh, N.: Agentic Text-to-SQL. (2026). <https://www.nirmalya.net/posts/2026/03/agentic-text-to-sql/>
8. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention is all you need. In: Advances in Neural Information Processing Systems, vol. 30 (2017)
9. Brown, T.B., Mann, B., Ryder, N., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020)
10. Yu, T., Zhang, R., Yang, K., et al.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proc. EMNLP, pp. 3911–3921 (2018)
11. Zhong, V., Xiong, C., Socher, R.: Seq2SQL: Generating structured queries from natural language using reinforcement learning. arXiv:1709.00103 (2017)
12. Rajkumar, N., Li, R., Bahdanau, D.: Evaluating the text-to-SQL capabilities of large language models. arXiv:2204.00498 (2022)
13. Pourreza, M., Rafiei, D.: DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In: Advances in Neural Information Processing Systems, vol. 36 (2023)
14. Gao, D., Wang, H., Li, Y., et al.: Text-to-SQL empowered by large language models: A benchmark evaluation. Proc. VLDB Endowment 17(5), 1132–1145 (2024)
15. Li, J., Hui, B., Qu, G., et al.: Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. In: Advances in Neural Information Processing Systems, vol. 36 (2023)
16. Katsogiannis-Meimarakis, G., Koutrika, G.: A survey on deep learning approaches for text-to-SQL. The VLDB Journal 32(4), 905–936 (2023)
17. Wang, B., Shin, R., Liu, X., Polozov, O., Richardson, M.: RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers. In: Proc. ACL, pp. 7567–7578 (2020)
18. Lei, W., Wang, W., Ma, Z., et al.: Re-examining the role of schema linking in text-to-SQL. In: Proc. EMNLP, pp. 6943–6954 (2020)
19. Lewis, P., Perez, E., Piktus, A., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474 (2020)
20. Yao, S., Zhao, J., Yu, D., et al.: ReAct: Synergizing reasoning and acting in language models. In: Proc. International Conference on Learning Representations (ICLR) (2023)

21. Schick, T., Dwivedi-Yu, J., Dessì, R., et al.: Toolformer: Language models can teach themselves to use tools. In: *Advances in Neural Information Processing Systems*, vol. 36 (2023)
22. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S.: Reflexion: Language agents with verbal reinforcement learning. In: *Advances in Neural Information Processing Systems*, vol. 36 (2023)
23. Wang, L., Ma, C., Feng, X., et al.: A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18(6), 186345 (2024)
24. Greshake, K., Abdelnabi, S., Mishra, S., et al.: Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pp. 79–90 (2023)
25. Perez, F., Ribeiro, I.: Ignore previous prompt: Attack techniques for language models. *arXiv:2211.09527* (2022)
26. OWASP Foundation: OWASP Top 10 for Large Language Model Applications (2025). <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
27. Sandhu, R.S., Coyne, E.J., Feinstein, H.L., Youman, C.E.: Role-based access control models. *Computer* 29(2), 38–47 (1996)
28. Ferraiolo, D.F., Sandhu, R., Gavrila, S., Kuhn, D.R., Chandramouli, R.: Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security* 4(3), 224–274 (2001)
29. Halfond, W.G.J., Viegas, J., Orso, A.: A classification of SQL injection attacks and countermeasures. In: *Proc. IEEE International Symposium on Secure Software Engineering (ISSSE)* (2006)